UT 03.1 – Renderizado condicional y listas en React con TypeScript

Índice

¿Qué son el contenido condicional y los datos de lista? **2**
Renderizar contenido de forma condicional **3**
Punto de partida: un botón y un texto oculto 3
Guardar elementos JSX en variables 4
Diferentes formas de renderizar contenido de forma condicional 5
¿Cuál es el mejor enfoque? 13
Salida de datos de listas **14**
Tipar los datos de la lista 14
Primer intento: un bucle for…of 15
Mapear datos: el método map() 16
Transformar antes de pintar: filtrar, ordenar y agrupar 18
Ejemplo integrador: catálogo de productos **20**
Errores frecuentes **23**
Resumen y conclusiones **24**
Preguntas de repaso **24**
Para saber más **25**

En los temas anteriores hemos aprendido los pilares de React (componentes, *props*, estado y eventos) y hemos visto cómo TypeScript nos ayuda a tipar cada uno de ellos. Con esas herramientas ya podemos construir casi cualquier interfaz, pero nos faltan dos piezas que aparecen en prácticamente todas las aplicaciones web: **mostrar contenido solo cuando se cumple una condición** (renderizado condicional) y **mostrar colecciones de datos** (listas).

Esta unidad (UT 03.1) se centra en **pintar** datos que ya tenemos. En la siguiente (UT 03.2) veremos cómo **modificar** las listas guardadas en el estado y cómo usar correctamente las claves (`key`).

Al terminar este tema serás capaz de:

- Mostrar u ocultar partes de la interfaz con `if`, ternarios, `&&`, mapas de valores y uniones discriminadas.
- Modelar con TypeScript los estados de una pantalla (cargando, error, datos) de forma que el compilador compruebe que los has tratado todos.
- Renderizar listas con `map()`, tipando tanto los datos como los componentes que los pintan.
- Filtrar, ordenar y agrupar datos antes de mostrarlos.
- Conocer las novedades recientes de React que afectan a estos temas (`<Activity>`, métodos de array no mutantes, `Object.groupBy`…).

En este tema utilizaremos las siguientes herramientas:

- **Visual Studio Code, Node.js y npm**: los mismos que en los temas anteriores.
- **Un proyecto Vite con React y TypeScript**, creado con `npm create vite@latest ejemplos-ut03-1 -- --template react-ts`.
- **Los ejemplos del tema** (`EJEM_03.1_TypeScript.zip`): un proyecto Vite listo para ejecutar con `npm install` y `npm run dev`, que incluye un menú para elegir y probar cada ejemplo. Cada bloque de código de este documento lleva una etiqueta (por ejemplo, *Ejemplo 01b*) que indica en qué archivo de la carpeta `src/ejemplos` lo puedes encontrar.

> **Novedad (2026): versiones de React y TypeScript**
>
> Todos los ejemplos de este tema se han compilado y probado con **React 19.2** y **TypeScript 6.0**, que es lo que instala hoy la plantilla `react-ts` de Vite (la plantilla actual incluye además la opción `erasableSyntaxOnly` y usa `oxlint` como linter).
>
> En octubre de 2025 se publicó **React 19.2**, que incorpora `<Activity>` (lo veremos en el renderizado condicional); en septiembre de 2026 llegó **React 19.3**.

¿Qué son el contenido condicional y los datos de lista?

Antes de ver las técnicas, conviene tener claro a qué nos referimos con cada término.

El **contenido condicional** es cualquier elemento de la interfaz (un botón, un mensaje, una sección entera) que solo debe mostrarse en determinadas circunstancias. Algunos ejemplos:

- Un mensaje de error que solo aparece si el usuario envía datos incorrectos en un formulario.
- Campos adicionales de un formulario que aparecen cuando el usuario elige «Soy empresa».
- Un indicador de carga mientras se piden datos a un *backend*.
- Un menú lateral que se despliega al pulsar el botón de la hamburguesa.

En el primer ejemplo, la condición es «el usuario ha introducido datos incorrectos» y el contenido condicional es el mensaje de error. Prácticamente todas las aplicaciones tienen elementos así.

Además, casi todas las aplicaciones muestran **listas de datos**. Una tienda online tiene una lista (o una cuadrícula, que es otro tipo de lista) de productos, una lista de artículos en el carrito, una lista de

pedidos y una lista de enlaces de navegación; un blog muestra entradas y comentarios; una agenda, eventos… Saber generar cualquier lista a partir de cualquier conjunto de datos es una habilidad imprescindible.

Y lo más habitual es que ambas cosas aparezcan juntas: **una lista que se puede filtrar, que muestra un mensaje cuando está vacía y que cambia el aspecto de sus elementos según su estado**. Esa combinación será el objetivo final del tema.

Renderizar contenido de forma condicional

Punto de partida: un botón y un texto oculto

Imagina que tienes un botón que, al pulsarlo, debe mostrar un cuadro de texto adicional. Inicialmente solo se ve el botón:

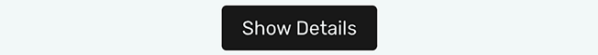


Figura 3.1.1 – Inicialmente solo aparece el botón en la pantalla

Tras pulsarlo, aparece el resumen de las condiciones de uso:

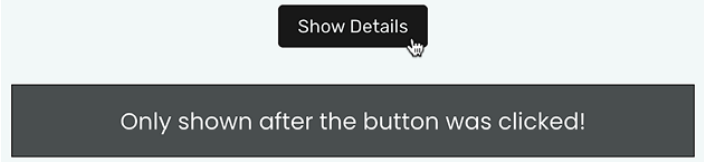


Figura 3.1.2 – Después de pulsar el botón se muestra el cuadro de información

Es un ejemplo sencillo, pero muy realista: la información nutricional de un plato en una web de comida a domicilio o las respuestas de una sección de preguntas frecuentes funcionan exactamente así.

Con lo que ya sabemos (estado y eventos), podemos escribir una primera versión. Fíjate en que `useState(false)` hace que TypeScript infiera el tipo `boolean` sin necesidad de escribirlo (lo vimos en la UT 02.3), y en que los nombres de las funciones manejadoras empiezan por `manejar…`:

Ejemplo 01a · CondicionesUso.tsx

```
import { useState } from "react";

export default function CondicionesUso() {
  const [mostrarCondiciones, setMostrarCondiciones] = useState(false);

  function manejarMostrarResumen() {
    setMostrarCondiciones(true);
  }

  let textoParrafo = "";
  if (mostrarCondiciones) {
    textoParrafo =
      "Al continuar, aceptas que no te indemnizaremos por ningún daño " +
      "o perjuicio causado por nuestros productos.";
  }

  return (
    <section>
      <button onClick={manejarMostrarResumen}>
        Mostrar resumen de las condiciones de uso
      </button>
      {/* ¡Este <p> existe siempre en el DOM, incluso vacío! */}
```

```
      <p>{textoParrafo}</p>
    </section>
  );
}
```

Este código ya tiene **valores** condicionales: `textoParrafo` cambia según el estado `mostrarCondiciones`. Pero el elemento `<p>` en sí **no** es condicional: siempre está en el DOM, aunque contenga una cadena vacía. Si inspeccionas la página con las herramientas de desarrollo verás un párrafo vacío:

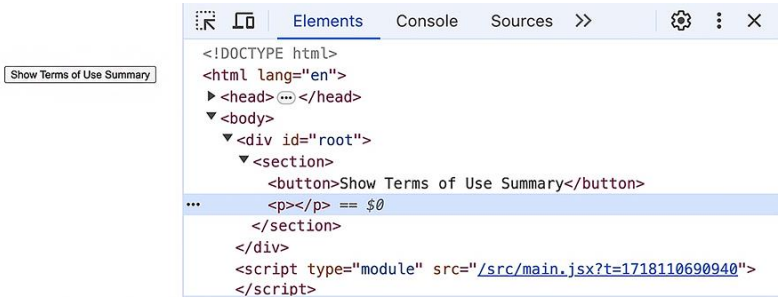


Figura 3.1.3 – Un párrafo vacío se renderiza como parte del DOM

Es probable que el impacto en el rendimiento sea mínimo, pero una página no se beneficia de tener elementos vacíos (además, un `<p>` vacío puede afectar a márgenes, estilos CSS y lectores de pantalla). Es mejor que el elemento **no exista** cuando no hace falta.

Guardar elementos JSX en variables

Podemos trasladar la idea de «valor condicional» a «elemento condicional». Además de textos y números, una variable puede almacenar **elementos JSX**, porque JSX es solo azúcar sintáctico: cada elemento JSX se convierte en una llamada a una función cuyo resultado es un objeto que se puede guardar en una variable, como cualquier otro valor.

En TypeScript, el tipo adecuado para una variable que a veces contiene JSX y a veces nada es **`ReactNode`**: engloba todo lo que React sabe pintar (elementos JSX, textos, números, `null`, `undefined`, booleanos y arrays de todos ellos).

Ejemplo 01b · CondicionesUso.tsx

```
import { useState, type ReactNode } from "react";

export default function CondicionesUso() {
  const [mostrarCondiciones, setMostrarCondiciones] = useState(false);

  function manejarMostrarResumen() {
    setMostrarCondiciones(true);
  }

  // ReactNode: cualquier cosa que React sepa pintar (JSX, texto, número, null...)
  let parrafo: ReactNode = null;
  if (mostrarCondiciones) {
    parrafo = (
      <p>
        Al continuar, aceptas que no te indemnizaremos por ningún daño o
        perjuicio causado por nuestros productos.
      </p>
    );
  }

  return (
    <section>
```

```
      <button onClick={manejarMostrarResumen}>
        Mostrar resumen de las condiciones de uso
      </button>
      {parrafo}
    </section>
  );
}
```

Si `mostrarCondiciones` es `false`, la variable `parrafo` vale `null` y React no renderiza nada en su lugar. Si es `true`, contiene el elemento `<p>` completo, que se inserta en el DOM al escribir `{parrafo}`. No estás limitado a un elemento: puedes guardar árboles JSX completos (varios elementos, anidados, hermanos…). Como regla general, **todo lo que puede devolver un componente se puede guardar en una variable**.

Conviene saber qué hace React con cada tipo de valor cuando lo encuentra dentro del JSX:

Valor dentro de { }	¿Se pinta algo?	Comentario
`true`, `false`	No	Permiten escribir `cond && <A />` sin que aparezca «false».
`null`, `undefined`	No	Son la forma habitual de «no renderizar nada».
`0`, `NaN`	**Sí** (muestra 0 o NaN)	Origen del error más típico con `&&` (lo veremos enseguida).
`""` (cadena vacía)	Nada visible	Se crea un nodo de texto vacío; no es un elemento.
`"Hola"`, `42`	Sí	Se muestran como texto.
Un array	Sí, cada elemento	Es la base para pintar listas.
Un objeto cualquiera	Error	«Objects are not valid as a React child».

> **Error frecuente con tipos: JSX.Element ya no existe como tipo global**
>
> En muchos tutoriales antiguos verás `JSX.Element` como tipo de retorno o de variable. Con las versiones actuales de `@types/react` (React 19) el espacio de nombres global `JSX` se ha eliminado y TypeScript da error. Usa `ReactElement` (un elemento concreto) o `ReactNode` (cualquier cosa renderizable); si necesitas el tipo de JSX, está disponible como `React.JSX.Element`.

Fragmento · tipos de JSX

```
// ❌ Con @types/react 19 ya no existe el espacio de nombres global JSX
// ❌ Error de TypeScript TS2503: Cannot find namespace 'JSX'.
const elementoMal: JSX.Element = <p>Hola</p>;

// ✅ Usa ReactElement (un elemento) o ReactNode (cualquier cosa renderizable)
import type { ReactElement } from "react";
const elementoBien: ReactElement = <p>Hola</p>;
```

Diferentes formas de renderizar contenido de forma condicional

Usar una variable y un `if` es una técnica válida y común, pero no es la única. React es JavaScript (y TypeScript) normal, así que sirve cualquier forma de elegir valores. Las más habituales son:

- Sentencias `if` y *returns* tempranos.
- El operador ternario `? :`.
- El operador lógico `&&` (cortocircuito).

- Mapas de valores (objetos o `Record`) y estructuras `switch`.
- Las novedades de React: `<Activity>` para ocultar sin desmontar.

Expresiones ternarias

Una **expresión ternaria** es una alternativa compacta al `if…else` cuando el objetivo es asignar un valor según una condición. Comparación directa:

```
let a = 1;
if (condicion) {
  a = 2;
}

// Equivalente con ternario
const a = condicion ? 2 : 1;
```

Aplicada a nuestro ejemplo, y escrita directamente dentro del JSX, queda así. Aprovechamos para mejorar el comportamiento: el botón ahora **alterna** (muestra y oculta) usando la forma funcional de `setState`, y añadimos `aria-expanded` para que los lectores de pantalla sepan si el contenido está desplegado.

Ejemplo 02 · CondicionesUso.tsx

```
import { useState } from "react";

export default function CondicionesUso() {
  const [mostrarCondiciones, setMostrarCondiciones] = useState(false);

  return (
    <section>
      <button
        onClick={() => setMostrarCondiciones((anterior) => !anterior)}
        aria-expanded={mostrarCondiciones}
      >
        {mostrarCondiciones ? "Ocultar" : "Mostrar"} resumen de las condiciones
        de uso
      </button>

      {mostrarCondiciones ? (
        <p>
          Al continuar, aceptas que no te indemnizaremos por ningún daño o
          perjuicio causado por nuestros productos.
        </p>
      ) : null}
    </section>
  );
}
```

Usamos `null` como alternativa porque se puede insertar con seguridad en el JSX: simplemente no se renderiza nada. Los ternarios ahorran líneas, pero **empeoran la legibilidad cuando se anidan**:

Fragmento · evita esto

```
const parrafoAnidado = !mostrar ? null : otraCondicion ? (
  <p>Condiciones de uso…</p>
) : null;
```

Este código es difícil de leer y de depurar. Cuando haya más de una condición, usa un `if` con *return* temprano:

Fragmento · mejor así

```
function textoCondiciones(): ReactNode {
  if (!mostrar || !otraCondicion) {
    return null;
  }
  return <p>Condiciones de uso…</p>;
}
```

El operador && (cortocircuito)

Muchos ternarios en JSX acaban teniendo esta forma:

```
{mostrarDetalles ? <h1>Detalles del producto</h1> : null}
```

Son innecesariamente largos: el caso `else` (`: null`) no aporta nada. Por eso es muy popular este otro patrón:

```
{mostrarDetalles && <h1>Detalles del producto</h1>}
```

Funciona porque `&&` devuelve el segundo operando si el primero es «verdadero» (*truthy*), y devuelve el primero si es «falso» (*falsy*). Esta técnica se llama **cortocircuito**. Como React no pinta `false`, el resultado es exactamente el deseado. Nuestro ejemplo con `&&`:

Ejemplo 03 · CondicionesUso.tsx

```
import { useState } from "react";

export default function CondicionesUso() {
  const [mostrarCondiciones, setMostrarCondiciones] = useState(false);

  return (
    <section>
      <button
        onClick={() => setMostrarCondiciones((anterior) => !anterior)}
        aria-expanded={mostrarCondiciones}
      >
        {mostrarCondiciones ? "Ocultar" : "Mostrar"} resumen de las condiciones
        de uso
      </button>

      {mostrarCondiciones && (
        <p>
          Al continuar, aceptas que no te indemnizaremos por ningún daño o
          perjuicio causado por nuestros productos.
        </p>
      )}
    </section>
  );
}
```

> **La trampa del cero**
>
> Con `&&` hay que tener cuidado si la condición **no es booleana**. Si `mensajes.length` vale `0`, la expresión `mensajes.length && <p>…</p>` devuelve `0`, y React **sí pinta el número 0** en pantalla. TypeScript no lo detecta porque `number && JSX` es una expresión perfectamente válida.
>
> Solución: haz que la condición sea siempre un booleano (`mensajes.length > 0`, `Boolean(valor)` o `!!valor`), o usa un *return* temprano. Algunas reglas de ESLint (por ejemplo `@typescript-eslint/strict-boolean-expressions`) pueden avisarte de estos casos.

Ejemplo 04 · TrampaDelCero.tsx

```tsx
import { useState } from "react";

interface PropsAvisos {
  mensajes: string[];
}

// ❌ Si mensajes.length vale 0, React pinta un "0" en pantalla
function AvisosConError({ mensajes }: PropsAvisos) {
  return <div>{mensajes.length && <p>Tienes {mensajes.length} avisos</p>}</div>;
}

// ✅ La condición es SIEMPRE un booleano
function AvisosCorrecto({ mensajes }: PropsAvisos) {
  return (
    <div>{mensajes.length > 0 && <p>Tienes {mensajes.length} avisos</p>}</div>
  );
}

// ✅ Alternativa: "salida temprana" (early return)
function AvisosSalidaTemprana({ mensajes }: PropsAvisos) {
  if (mensajes.length === 0) {
    return null;
  }
  return <p>Tienes {mensajes.length} avisos</p>;
}

export default function TrampaDelCero() {
  const [mensajes, setMensajes] = useState<string[]>([]);

  return (
    <section>
      <button onClick={() => setMensajes((ant) => [...ant, "Nuevo aviso"])}>
        Añadir aviso
      </button>
      <button onClick={() => setMensajes([])}>Vaciar</button>
      <h3>Con error</h3>
      <AvisosConError mensajes={mensajes} />
      <h3>Correcto</h3>
      <AvisosCorrecto mensajes={mensajes} />
      <h3>Salida temprana</h3>
      <AvisosSalidaTemprana mensajes={mensajes} />
    </section>
  );
}
```

La última variante (`AvisosSalidaTemprana`) usa una **salida temprana**: si no hay nada que mostrar, el componente devuelve `null` y termina. Es un patrón muy legible cuando la condición afecta al componente completo.

Mapas de valores: ¡sé creativo!

Como el código React es JavaScript normal, cualquier enfoque que seleccione valores según una condición vale. Cuando tienes un conjunto de casos que dependen de una clave, a menudo es más limpio un **objeto que haga de mapa** que varios `if` o ternarios encadenados. Aquí TypeScript nos da mucho más que JavaScript: con `Record<Clave, Valor>` obligamos a que el mapa tenga **todas** las claves posibles, y con una unión de textos limitamos qué claves se pueden usar.

Ejemplo 05 · SelectorIdioma.tsx

```tsx
import { useState } from "react";

type Pais = "de" | "us" | "uk";

// Record<Pais, string> obliga a definir TODAS las claves de Pais
const idiomas: Record<Pais, string> = {
  de: "de-DE",
  us: "en-US",
  uk: "en-GB",
};

interface PropsSelectorIdioma {
  pais: Pais;
}

function SelectorIdioma({ pais }: PropsSelectorIdioma) {
  return <p>Idioma seleccionado: {idiomas[pais]}</p>;
}

export default function DemoSelectorIdioma() {
  const [pais, setPais] = useState<Pais>("de");

  return (
    <section>
      <select value={pais} onChange={(e) => setPais(e.target.value as Pais)}>
        {(Object.keys(idiomas) as Pais[]).map((codigo) => (
          <option key={codigo} value={codigo}>
            {codigo}
          </option>
        ))}
      </select>
      <SelectorIdioma pais={pais} />
    </section>
  );
}
```

El componente muestra `de-DE`, `en-US` o `en-GB` según la propiedad `pais`. Esto se consigue con la **notación de corchetes**: en lugar de `idiomas.de`, escribimos `idiomas[pais]`, donde `pais` es una variable. Si añades un nuevo país al tipo `Pais` y olvidas añadirlo al mapa, TypeScript te lo marca como error.

> **Aquí JavaScript lo deja pasar y TypeScript no**
>
> En el ejemplo original (JavaScript), `languages[country]` funcionaba con cualquier valor. En TypeScript, si `pais` fuera un `string` cualquiera, el compilador se queja porque no puede garantizar que ese texto sea una clave válida del objeto. Por eso conviene tipar `pais` con la unión `"de" | "us" | "uk"`.

Fragmento · el error TS7053

```tsx
const idiomasBasico = { de: "de-DE", us: "en-US", uk: "en-GB" };

function SelectorMal({ pais }: { pais: string }) {
  // ❌ Error de TypeScript TS7053: Element implicitly has an 'any' type (un string no
sirve de índice)
  return <p>Idioma seleccionado: {idiomasBasico[pais]}</p>;
}
```

Estados de pantalla con uniones discriminadas

Uno de los usos más importantes del renderizado condicional es mostrar una cosa u otra según el **estado de una petición**: cargando, error o datos disponibles. Una solución habitual en JavaScript es usar tres variables sueltas (`cargando`, `error`, `datos`), pero eso permite estados imposibles, como «cargando y con error a la vez».

TypeScript permite modelar el problema con una **unión discriminada**: un tipo que es una de varias formas posibles, y donde una propiedad común (aquí, `estado`) indica de cuál se trata. Al comprobar esa propiedad con un `switch` o un `if`, TypeScript **reduce** el tipo y solo permite acceder a las propiedades que existen en ese caso.

Ejemplo 06 · EstadoPeticion.tsx

```
import { useState } from "react";

// Unión discriminada: la propiedad "estado" decide qué otras propiedades existen
type EstadoPeticion<T> =
  | { estado: "cargando" }
  | { estado: "error"; mensaje: string }
  | { estado: "correcto"; datos: T };

interface PropsListaUsuarios {
  peticion: EstadoPeticion<string[]>;
}

function ListaUsuarios({ peticion }: PropsListaUsuarios) {
  switch (peticion.estado) {
    case "cargando":
      return <p>Cargando usuarios…</p>;
    case "error":
      // Aquí TypeScript sabe que existe peticion.mensaje
      return <p role="alert">Error: {peticion.mensaje}</p>;
    case "correcto":
      // Aquí TypeScript sabe que existe peticion.datos
      return (
        <ul>
          {peticion.datos.map((nombre) => (
            <li key={nombre}>{nombre}</li>
          ))}
        </ul>
      );
    default: {
      // Si añades un estado nuevo y olvidas tratarlo, esta línea no compila
      const _comprobacion: never = peticion;
      return _comprobacion;
    }
  }
}

export default function DemoEstadoPeticion() {
  const [peticion, setPeticion] = useState<EstadoPeticion<string[]>>({
    estado: "cargando",
  });

  return (
    <section>
      <button onClick={() => setPeticion({ estado: "cargando" })}>
        Cargando
      </button>
      <button
        onClick={() =>
```

```
          setPeticion({ estado: "error", mensaje: "No se pudo conectar" })
        }
      >
        Error
      </button>
      <button
        onClick={() =>
          setPeticion({ estado: "correcto", datos: ["Ana", "Luis", "Marta"] })
        }
      >
        Correcto
      </button>
      <ListaUsuarios peticion={peticion} />
    </section>
  );
}
```

Fíjate en el `default`: asignar `peticion` a una variable de tipo `never` solo compila si **no queda ningún caso sin tratar**. Si algún día añades un cuarto estado (por ejemplo, `"vacio"`) y olvidas tratarlo en el `switch`, el compilador marcará esa línea como error. Este patrón (*exhaustividad*) lo usaremos más adelante cuando pidamos datos a una API y cuando trabajemos con los estados de carga de Next.js.

Elegir la etiqueta o el componente de forma condicional

A veces no se trata de mostrar u ocultar, sino de elegir **qué etiqueta** usar. Ocurre al crear componentes que envuelven a los nativos: por ejemplo, un componente que debe renderizarse como `<button>` o como `<a>` según el caso. Esta es la versión tipada con una unión discriminada (comprobamos `esBoton` con un `if`, igual que en el estado de la petición):

Ejemplo 07 · BotonOEnlace.tsx

```
import type { ComponentPropsWithoutRef, ElementType, ReactNode } from "react";

// ── Versión 1: con `if` y unión discriminada (la más segura) ──
type PropsComoBoton = {
  esBoton: true;
  config?: ComponentPropsWithoutRef<"button">;
  children: ReactNode;
};

type PropsComoEnlace = {
  esBoton: false;
  config: ComponentPropsWithoutRef<"a">; // href, target, rel...
  children: ReactNode;
};

function BotonOEnlace(props: PropsComoBoton | PropsComoEnlace) {
  if (props.esBoton) {
    return <button {...props.config}>{props.children}</button>;
  }
  return <a {...props.config}>{props.children}</a>;
}

// ── Versión 2: etiqueta dinámica (menos código, menos tipado) ──
interface PropsEtiqueta {
  esBoton: boolean;
  config?: Record<string, unknown>;
  children: ReactNode;
}
```

```
function BotonOEnlaceCorto({ esBoton, config, children }: PropsEtiqueta) {
  // ElementType = "cualquier etiqueta o componente"; empieza por mayúscula
  const Etiqueta: ElementType = esBoton ? "button" : "a";
  return <Etiqueta {...config}>{children}</Etiqueta>;
}

export default function DemoBotonOEnlace() {
  return (
    <section>
      <BotonOEnlace esBoton config={{ onClick: () => alert("¡Hola!") }}>
        Soy un botón
      </BotonOEnlace>{" "}
      <BotonOEnlace esBoton={false} config={{ href: "https://react.dev" }}>
        Soy un enlace
      </BotonOEnlace>{" "}
      <BotonOEnlaceCorto esBoton={false} config={{ href: "https://react.dev" }}>
        Enlace (versión corta)
      </BotonOEnlaceCorto>
    </section>
  );
}
```

Puntos a destacar:

- `ComponentPropsWithoutRef<"button">` y `ComponentPropsWithoutRef<"a">` son tipos de React que describen **todas** las propiedades válidas de un `<button>` o un `<a>` (`onClick`, `disabled`, `href`, `target`…).
- El **operador de propagación** (`{...config}`) convierte cada pareja clave-valor del objeto en una propiedad del elemento: `<a {...{href: "https://react.dev"}}>` equivale a `<a href="https://react.dev">`. (En el material original este ejemplo usaba una propiedad `link`, pero la propiedad HTML correcta de un enlace es `href`.)
- En la versión corta, una **constante que empieza por mayúscula** (`Etiqueta`) puede contener el nombre de una etiqueta HTML como texto (`"button"`, `"a"`) o incluso una función de componente, y se usa en el JSX como si fuera un componente. Su tipo es `ElementType`. La contrapartida es que se pierde la comprobación fina de las propiedades, por eso en la versión corta `config` es un simple `Record<string, unknown>`.

> **Novedad (React 19): ref como propiedad normal**
>
> Desde React 19, `ref` se puede recibir como una propiedad más en los componentes de función, así que los componentes «envoltorio» ya no necesitan `forwardRef` en la mayoría de los casos. Esto simplifica la escritura de componentes que reenvían sus propiedades a un `<button>` o un `<input>` nativo.

Ocultar sin desmontar: <Activity> (React 19.2)

Con todas las técnicas anteriores, cuando la condición es falsa el componente **desaparece**: React lo desmonta y pierde su estado interno. Si el usuario escribía un borrador en un `<textarea>`, cambiaba de pestaña y volvía, el texto ya no está. Hasta ahora la solución era subir el estado a un componente padre, o ocultar con CSS (`display: none`) manualmente.

> **Novedad (2025-2026): el componente <Activity>**
>
> React 19.2 (octubre de 2025) incorporó `<Activity>`, que permite **ocultar una parte de la interfaz conservando su estado**. Admite una propiedad `mode` con dos valores: `"visible"` (por defecto) y `"hidden"`.
>
> • Al ocultarlo, React aplica `display: none` a los hijos, **conserva su estado** (incluido el del DOM, como el texto de un `<textarea>`) y **destruye sus efectos** (las suscripciones se limpian).
> • Al volver a mostrarlo, restaura el estado anterior y vuelve a crear los efectos.

> • El contenido oculto se sigue actualizando, pero con menor prioridad. También permite **pre-renderizar** contenido que aún no se ve.
> • Ojo: un elemento `<video>` o `<audio>` oculto sigue reproduciéndose; hay que pararlo en una función de limpieza de un efecto.

Ejemplo 08 · Activity.tsx

```
import { Activity, useState } from "react";

type Pestana = "borrador" | "ayuda";

function Borrador() {
  return <textarea placeholder="Escribe aquí tu borrador…" rows={4} />;
}

// Renderizado condicional clásico: al ocultar, el componente se DESMONTA
function PestanasCondicional() {
  const [activa, setActiva] = useState<Pestana>("borrador");
  return (
    <section>
      <h3>Con renderizado condicional</h3>
      <button onClick={() => setActiva("borrador")}>Borrador</button>
      <button onClick={() => setActiva("ayuda")}>Ayuda</button>
      {activa === "borrador" && <Borrador />}
      {activa === "ayuda" && <p>Texto de ayuda</p>}
    </section>
  );
}

// <Activity>: al ocultar, React conserva el estado y el DOM
function PestanasActivity() {
  const [activa, setActiva] = useState<Pestana>("borrador");
  return (
    <section>
      <h3>Con &lt;Activity&gt;</h3>
      <button onClick={() => setActiva("borrador")}>Borrador</button>
      <button onClick={() => setActiva("ayuda")}>Ayuda</button>
      <Activity mode={activa === "borrador" ? "visible" : "hidden"}>
        <Borrador />
      </Activity>
      <Activity mode={activa === "ayuda" ? "visible" : "hidden"}>
        <p>Texto de ayuda</p>
      </Activity>
    </section>
  );
}

export default function DemoActivity() {
  return (
    <>
      <PestanasCondicional />
      <PestanasActivity />
    </>
  );
}
```

Prueba este ejemplo: escribe algo en el borrador de cada bloque, cambia a «Ayuda» y vuelve. Con renderizado condicional el borrador se pierde; con `<Activity>` se conserva. Úsalo para pestañas, paneles laterales o asistentes de varios pasos donde no quieras perder lo que el usuario ya ha

rellenado. Para el resto de casos (la mayoría), el renderizado condicional clásico sigue siendo la opción adecuada.

¿Cuál es el mejor enfoque?

No hay una respuesta única: depende del problema y de las preferencias del equipo. Como guía general, puedes seguir el siguiente esquema y la tabla resumen:

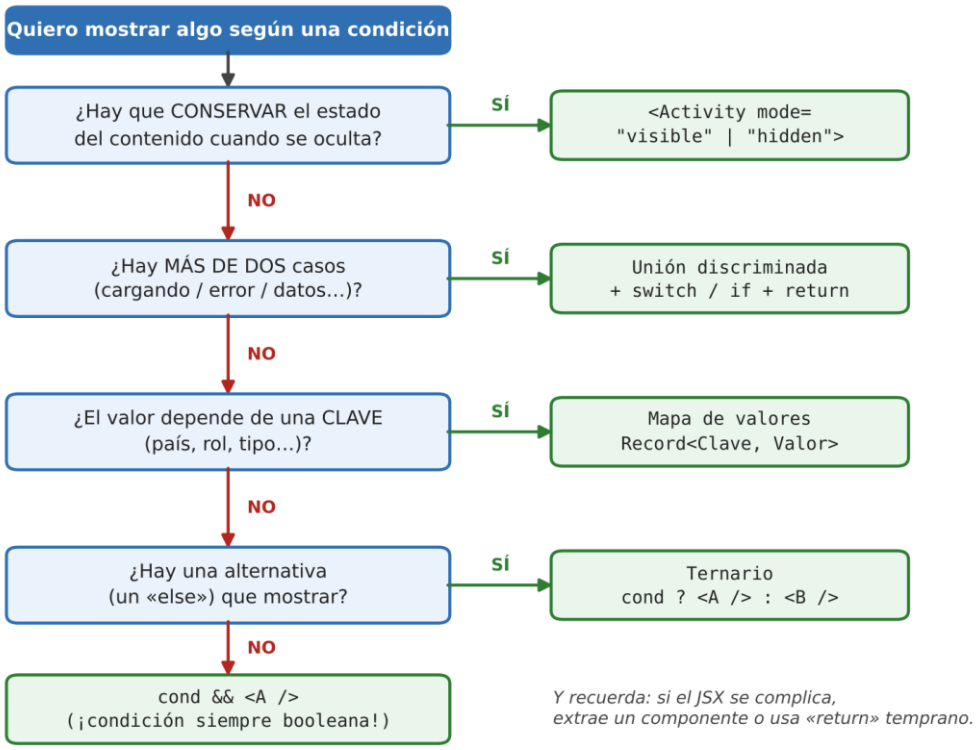


Figura 3.1.4 – Guía para elegir la técnica de renderizado condicional

Técnica	Úsala cuando…	Ten cuidado con…
`if` + *return* temprano	La condición afecta a todo el componente o hay varios casos.	Que el componente quede con muchos `return`; extrae subcomponentes.
Variable con JSX	El fragmento condicional es largo o se calcula con varios pasos.	No dejar variables sin inicializar: tipa como `ReactNode`.
Ternario `? :`	Hay exactamente dos alternativas.	Anidarlos: ilegible. Mejor `if`.
`&&`	Solo hay «mostrar o no mostrar».	Condiciones numéricas (`0`) o cadenas: conviértelas a booleano.
Mapa (`Record`)	El valor depende de una clave (país, rol, tipo, estado).	Tipar la clave con una unión para que TypeScript compruebe todas.
Unión discriminada + `switch`	Estados mutuamente excluyentes (cargando / error / datos).	Añadir el `default` con `never` para exigir exhaustividad.
`<Activity>`	Quieres ocultar conservando el estado.	Efectos que se destruyen al ocultar; medios que siguen sonando.

Salida de datos de listas

Además del contenido condicional, casi siempre tendrás que mostrar **listas de datos**: productos, pedidos, usuarios, elementos de navegación… En una aplicación React, esos datos suelen llegar como un **array** (por ejemplo, recibido por *props* desde un componente que los ha pedido a una API del *backend*).

Tipar los datos de la lista

Lo primero en TypeScript es describir la forma de cada elemento. Para objetos usamos una `interface` (o un `type`). Y si un campo solo admite un conjunto cerrado de valores, como una categoría, lo mejor es partir de un array `as const` y **derivar el tipo** de él: así hay una única fuente de verdad.

Archivo comun/productos.ts (lo usan los ejemplos siguientes)

```
export const categorias = ["libros", "hogar", "oficina", "cursos"] as const;

export type Categoria = (typeof categorias)[number];

export interface Producto {
  id: string;
  titulo: string;
  precio: number;
  categoria: Categoria;
}

export const productos: Producto[] = [
  { id: "p1", titulo: "Un libro", precio: 59.99, categoria: "libros" },
  { id: "p2", titulo: "Una alfombra", precio: 129.49, categoria: "hogar" },
  { id: "p3", titulo: "Otro libro", precio: 39.99, categoria: "libros" },
  { id: "p4", titulo: "Lámpara de mesa", precio: 24.9, categoria: "hogar" },
  {
    id: "p5",
    titulo: "Manual de TypeScript",
    precio: 34.5,
    categoria: "libros",
  },
  {
    id: "p6",
    titulo: "Silla ergonómica",
    precio: 269.99,
    categoria: "oficina",
  },
  {
    id: "p7",
    titulo: "Escritorio elevable",
    precio: 349,
    categoria: "oficina",
  },
  { id: "p8", titulo: "Curso de React", precio: 19.99, categoria: "cursos" },
];

const formateadorEuros = new Intl.NumberFormat("es-ES", {
  style: "currency",
  currency: "EUR",
});

export function formatearPrecio(valor: number): string {
  return formateadorEuros.format(valor);
}
```

Fíjate en tres detalles: `(typeof categorias)[number]` genera la unión `"libros" | "hogar" | "oficina" | "cursos"` a partir del array; el precio se formatea con `Intl.NumberFormat`; y los identificadores son cadenas (`"p1"`, `"p2"`…), como suele ocurrir con los datos que vienen de una base de datos.

Primer intento: un bucle for…of

Una forma muy didáctica de entender la salida de listas es olvidarnos de React y transformar los datos con JavaScript normal. Queremos pasar de un array de objetos `Producto` a un array de elementos JSX, uno por producto. Con un bucle:

Ejemplo 09 · ListaProductos.tsx

```
import type { ReactElement } from "react";
import { productos, formatearPrecio, type Producto } from "./comun/productos";

interface PropsListaProductos {
  productos: Producto[];
}

function ListaProductos({ productos }: PropsListaProductos) {
  const elementos: ReactElement[] = [];

  for (const producto of productos) {
    elementos.push(
      <li>
        <h2>{producto.titulo}</h2>
        <p>{formatearPrecio(producto.precio)}</p>
      </li>,
    );
  }

  return <ul>{elementos}</ul>;
}

export default function DemoListaFor() {
  return <ListaProductos productos={productos} />;
}
```

Puede parecer extraño ver JSX dentro de un `push()`, pero el JSX se puede usar en cualquier sitio donde se pueda usar un valor normal. Lo declaramos como `ReactElement[]`. Y la segunda pieza clave es que **JSX acepta arrays**: al insertar `{elementos}`, cada elemento del array se renderiza uno junto a otro. Estos dos fragmentos producen el mismo resultado:

```
return <div>{[<p>Hola</p>, <p>Otro elemento</p>]}</div>;

return (
  <div>
    <p>Hola</p>
    <p>Otro elemento</p>
  </div>
);
```

Es un enfoque válido, pero no el más habitual en proyectos reales.

Mapear datos: el método map()

Igual que con los `if` y los ternarios, podemos sustituir el bucle por algo más corto. JavaScript ofrece `map()`, un método de todos los arrays que **ejecuta una función para cada elemento y devuelve un array nuevo con los resultados**. En TypeScript, el tipo del resultado se infiere automáticamente:

Fragmento · map() con tipos inferidos

```
const usuarios = [
  { id: "u1", nombre: "Max", edad: 35 },
  { id: "u2", nombre: "Ana", edad: 32 },
];

// TypeScript infiere que nombres es string[]
const nombres = usuarios.map((usuario) => usuario.nombre);
```

Como `map()` puede generar un array de elementos JSX, nuestro componente se reduce a:

Ejemplo 10a · ListaProductos.tsx

```
import { productos, formatearPrecio, type Producto } from "./comun/productos";

interface PropsListaProductos {
  productos: Producto[];
}

function ListaProductos({ productos }: PropsListaProductos) {
  return (
    <ul>
      {productos.map((producto) => (
        <li>
          <h2>{producto.titulo}</h2>
          <p>{formatearPrecio(producto.precio)}</p>
        </li>
      ))}
    </ul>
  );
}

export default function DemoListaMap() {
  return <ListaProductos productos={productos} />;
}
```

Observa que el parámetro `producto` de la función flecha **no necesita tipo**: TypeScript sabe que `productos` es `Producto[]` y deduce que cada elemento es un `Producto`. Si escribes `producto.` el editor te sugerirá solo `id`, `titulo`, `precio` y `categoria`, y si te equivocas al escribir `producto.nombre` te marcará el error al momento.

`map()` en línea (directamente dentro del JSX) es, de facto, el método estándar para pintar listas en React. Si la transformación es compleja (varias líneas de cálculo, JSX muy anidado), conviene **extraer un componente** para cada elemento, que además queda mucho más reutilizable y fácil de tipar:

Ejemplo 10b · ListaProductos.tsx

```
import { productos, formatearPrecio, type Producto } from "./comun/productos";

interface PropsProductoItem {
  producto: Producto;
}

function ProductoItem({ producto }: PropsProductoItem) {
  return (
    <li>
```

```
      <h2>{producto.titulo}</h2>
      <p>{formatearPrecio(producto.precio)}</p>
    </li>
  );
}

interface PropsListaProductos {
  productos: readonly Producto[]; // readonly: este componente no modifica el array
}

function ListaProductos({ productos }: PropsListaProductos) {
  if (productos.length === 0) {
    return <p>No hay productos que mostrar.</p>;
  }

  return (
    <ul>
      {productos.map((producto) => (
        <ProductoItem key={producto.id} producto={producto} />
      ))}
    </ul>
  );
}

export default function DemoListaComponente() {
  return (
    <>
      <ListaProductos productos={productos} />
      <ListaProductos productos={[]} />
    </>
  );
}
```

- `ProductoItem` recibe **un** producto y se ocupa de cómo se pinta. `ListaProductos` recibe **la lista** y se ocupa de recorrerla.
- `readonly Producto[]` indica que el componente solo lee el array y no lo modifica; acepta tanto `Producto[]` como `readonly Producto[]`.
- Cuando la lista está vacía, mostramos un mensaje en vez de un `<ul>` sin elementos. **Piensa siempre en el caso «no hay datos»**: es un renderizado condicional más.
- Ya hemos añadido `key={producto.id}`. Por qué es imprescindible se explica en la UT 03.2 (el ejemplo 10a, sin `key`, provoca una advertencia en la consola); de momento, escribe siempre `key={elemento.id}`.

Transformar antes de pintar: filtrar, ordenar y agrupar

Los datos de una lista rara vez se muestran tal cual: se filtran por una búsqueda, se ordenan, se agrupan… Todas esas operaciones se resuelven con métodos de array **encadenados** antes del `map()`:

```
const visibles = productos
  .filter((p) => p.categoria === "libros")   // me quedo con los libros
  .toSorted((a, b) => a.precio - b.precio)   // ordeno por precio (sin mutar)
  .map((p) => <ProductoItem key={p.id} producto={p} />);
```

> **Buena práctica: los datos derivados no se guardan en el estado**
>
> Si la lista visible se puede **calcular** a partir del estado (la lista completa + el texto de búsqueda + el orden), no la guardes en otro `useState`: calcúlala en cada renderizado. Así evitas tener dos copias de los datos que se desincronizan. El estado guarda lo mínimo; todo lo demás se deriva.

Métodos de array «no mutantes» (ES2023)

En el ejemplo anterior hemos usado `toSorted()` en lugar de `sort()`. Es una de las incorporaciones de ES2023 que más ayudan en React: `sort()` y `reverse()` **modifican el array original**, mientras que `toSorted()` y `toReversed()` devuelven una copia. `reverse()` frente a `toReversed()`:

```
const aficiones = ["Deportes", "Cocina"];
const invertidas = aficiones.reverse();  // ⚠ ¡también modifica "aficiones"!
```

Con la versión no mutante, el original queda intacto:

Fragmento · toReversed()

```
const aficiones = ["Deporte", "Cocina"];

const invertidas = aficiones.toReversed(); // ["Cocina", "Deporte"]
// aficiones sigue siendo ["Deporte", "Cocina"]: no se ha modificado
```

En la UT 03.2 verás la tabla completa de métodos mutantes y sus alternativas. Para poder usarlos, la opción `lib` de `tsconfig.json` debe incluir `ES2023` o superior (la plantilla actual de Vite ya lo hace) y el navegador debe ser razonablemente moderno (todos los actuales lo son).

Agrupar con Object.groupBy (ES2024)

Otra necesidad frecuente es mostrar una lista **agrupada** (productos por categoría, tareas por prioridad). `Object.groupBy()` devuelve un objeto cuyas claves son los grupos:

Ejemplo 11 · AgruparPorCategoria.tsx

```
import { categorias, productos } from "./comun/productos";

export default function AgruparPorCategoria() {
  // Object.groupBy (ES2024) → Partial<Record<Categoria, Producto[]>>
  const grupos = Object.groupBy(productos, (p) => p.categoria);

  return (
    <>
      {categorias.map((categoria) => {
        const delGrupo = grupos[categoria];
        if (!delGrupo) return null; // categoría sin productos
        return (
          <section key={categoria}>
            <h2>{categoria}</h2>
            <ul>
              {delGrupo.map((p) => (
                <li key={p.id}>{p.titulo}</li>
              ))}
            </ul>
          </section>
        );
      })}
    </>
  );
}
```

> **Nota de configuración**
>
> `Object.groupBy` pertenece a ES2024. Para que TypeScript lo reconozca, en `tsconfig.app.json` pon `"lib": ["ES2024", "DOM"]`. Su tipo de retorno es `Partial<Record<Categoria, Producto[]>>`: por eso en el ejemplo comprobamos `if (!delGrupo)` antes de usarlo, ya que una categoría sin productos no aparece como clave.

Elementos «nulos» en una lista (TypeScript 5.5)

Es habitual tener arrays que mezclan datos y `null` (por ejemplo, el resultado de buscar varios elementos). Desde TypeScript 5.5, el compilador **infiere automáticamente los predicados de tipo**, de modo que `filter()` elimina el `null` también del tipo, sin tener que escribir `(u): u is Usuario => …`:

Ejemplo 12 · FiltrarNulos.ts

```ts
interface Usuario {
  nombre: string;
}

const respuesta: (Usuario | null)[] = [
  { nombre: "Ana" },
  null,
  { nombre: "Luis" },
];

// Desde TypeScript 5.5 el compilador INFIERE el predicado de tipo:
// `conNombre` es Usuario[] (sin null) sin escribir `(u): u is Usuario => ...`
const conNombre = respuesta.filter((u) => u !== null);

const nombres: string[] = conNombre.map((u) => u.nombre);
```

Varios elementos por cada dato: Fragment con key

A veces cada elemento de la lista necesita pintar **varios nodos hermanos** (por ejemplo, `<dt>` y `<dd>` en una lista de definiciones). El fragmento corto `<>…</>` no admite la propiedad `key`, así que hay que usar `<Fragment key={…}>`:

Ejemplo 13 · Glosario.tsx

```tsx
import { Fragment } from "react";

interface Termino {
  palabra: string;
  definicion: string;
}

const glosario: Termino[] = [
  {
    palabra: "JSX",
    definicion: "Sintaxis que mezcla marcado y JavaScript/TypeScript.",
  },
  {
    palabra: "key",
    definicion: "Identificador estable de cada elemento de una lista.",
  },
];

export default function Glosario() {
  return (
    <dl>
      {glosario.map((t) => (
        // <>...</> no admite `key`; <Fragment key=...> sí
        <Fragment key={t.palabra}>
          <dt>{t.palabra}</dt>
          <dd>{t.definicion}</dd>
        </Fragment>
      ))}
    </dl>
```

```
  );
}
```

Ejemplo integrador: catálogo de productos

Terminamos con un ejemplo que combina todo lo visto: una lista de productos que se puede **buscar**, **filtrar por categoría** y **ordenar**, con renderizado condicional (mensaje de lista vacía, etiqueta «Económico», contador de resultados), tipos para las *props*, estado mínimo y datos derivados. Está dividido en varios archivos, como se haría en un proyecto real:

- `ordenar.ts` — el tipo `Orden` y un `Record` con las funciones de comparación.
- `Filtros.tsx` — el formulario de filtros (componente «controlado» desde el padre).
- `TarjetaProducto.tsx` y `ListaProductos.tsx` — cómo se pinta un producto y la lista.
- `Catalogo.tsx` — guarda el estado y calcula la lista visible.

Ejemplo 14 · ordenar.ts

```
import type { Producto } from "../comun/productos";

export type Orden = "titulo" | "precio-asc" | "precio-desc";

// Record<Orden, ...> garantiza que existe un comparador para cada criterio
export const comparadores: Record<Orden, (a: Producto, b: Producto) => number> =
  {
    titulo: (a, b) => a.titulo.localeCompare(b.titulo, "es"),
    "precio-asc": (a, b) => a.precio - b.precio,
    "precio-desc": (a, b) => b.precio - a.precio,
  };
```

Ejemplo 14 · Filtros.tsx

```
import { categorias, type Categoria } from "../comun/productos";
import type { Orden } from "./ordenar";

export type FiltroCategoria = Categoria | "todas";

interface PropsFiltros {
  busqueda: string;
  categoria: FiltroCategoria;
  orden: Orden;
  alCambiarBusqueda: (valor: string) => void;
  alCambiarCategoria: (valor: FiltroCategoria) => void;
  alCambiarOrden: (valor: Orden) => void;
}

export default function Filtros({
  busqueda,
  categoria,
  orden,
  alCambiarBusqueda,
  alCambiarCategoria,
  alCambiarOrden,
}: PropsFiltros) {
  return (
    <form onSubmit={(e) => e.preventDefault()}>
      <input
        type="search"
        value={busqueda}
```

```
        onChange={(e) => alCambiarBusqueda(e.target.value)}
        placeholder="Buscar…"
      />

      <select
        value={categoria}
        onChange={(e) => alCambiarCategoria(e.target.value as FiltroCategoria)}
      >
        <option value="todas">Todas</option>
        {categorias.map((c) => (
          <option key={c} value={c}>
            {c}
          </option>
        ))}
      </select>

      <select
        value={orden}
        onChange={(e) => alCambiarOrden(e.target.value as Orden)}
      >
        <option value="titulo">Título</option>
        <option value="precio-asc">Precio ↑</option>
        <option value="precio-desc">Precio ↓</option>
      </select>
    </form>
  );
}
```

Ejemplo 14 · TarjetaProducto.tsx

```
import { formatearPrecio, type Producto } from "../comun/productos";

interface PropsTarjetaProducto {
  producto: Producto;
}

export default function TarjetaProducto({ producto }: PropsTarjetaProducto) {
  const esEconomico = producto.precio < 30;

  return (
    <li>
      <h3>
        {producto.titulo}{" "}
        {esEconomico && <small style={{ color: "green" }}>¡Económico!</small>}
      </h3>
      <p>
        {formatearPrecio(producto.precio)} · {producto.categoria}
      </p>
    </li>
  );
}
```

Ejemplo 14 · ListaProductos.tsx

```
import type { Producto } from "../comun/productos";
import TarjetaProducto from "./TarjetaProducto";

interface PropsListaProductos {
  productos: readonly Producto[];
}
```

```
export default function ListaProductos({ productos }: PropsListaProductos) {
  if (productos.length === 0) {
    return <p>No hay productos que coincidan con los filtros.</p>;
  }

  return (
    <ul>
      {productos.map((producto) => (
        <TarjetaProducto key={producto.id} producto={producto} />
      ))}
    </ul>
  );
}
```

Ejemplo 14 · Catalogo.tsx

```
import { useState } from "react";
import { productos } from "../comun/productos";
import Filtros, { type FiltroCategoria } from "./Filtros";
import ListaProductos from "./ListaProductos";
import { comparadores, type Orden } from "./ordenar";

export default function Catalogo() {
  const [busqueda, setBusqueda] = useState("");
  const [categoria, setCategoria] = useState<FiltroCategoria>("todas");
  const [orden, setOrden] = useState<Orden>("titulo");

  // Se calcula en cada render a partir del estado: NO es otro useState
  const texto = busqueda.trim().toLowerCase();
  const visibles = productos
    .filter((p) => categoria === "todas" || p.categoria === categoria)
    .filter((p) => p.titulo.toLowerCase().includes(texto))
    .toSorted(comparadores[orden]);

  return (
    <section>
      <Filtros
        busqueda={busqueda}
        categoria={categoria}
        orden={orden}
        alCambiarBusqueda={setBusqueda}
        alCambiarCategoria={setCategoria}
        alCambiarOrden={setOrden}
      />
      <p>
        Mostrando {visibles.length} de {productos.length} productos
      </p>
      <ListaProductos productos={visibles} />
    </section>
  );
}
```

Observa cómo se reparten las responsabilidades: `Catalogo` es el único que tiene estado; `Filtros` recibe valores y **funciones** (`alCambiarBusqueda`…) por *props* y no sabe nada de la lista; y `ListaProductos` no sabe de dónde vienen los productos ni cómo se han filtrado. Si cambias el tipo `Orden` (por ejemplo, añadiendo `"categoria"`), TypeScript te señalará en qué sitios falta tratar el nuevo caso (el `Record` de comparadores).

Errores frecuentes

Síntoma	Causa probable	Solución
Aparece un `0` en pantalla.	`cantidad && <Componente />` con `cantidad = 0`.	`cantidad > 0 && …` o un ternario.
TypeScript: «Cannot find namespace 'JSX'».	Tipo `JSX.Element` con `@types/react` 19.	`ReactElement`, `ReactNode` o `React.JSX.Element`.
TypeScript: «Element implicitly has an 'any' type…» al indexar un objeto.	Se indexa un objeto con un `string` genérico.	Tipa la clave con una unión y el objeto con `Record<Clave, Valor>`.
Un `<p>` o `<div>` vacío en el DOM.	El elemento no es condicional, solo su contenido.	Haz condicional el elemento completo (`&&`, ternario o variable JSX).
Ternarios anidados ilegibles.	Varias condiciones en una sola expresión.	Usa `if` con *return* temprano o extrae un componente.
La lista filtrada no coincide con la lista completa.	Se ha guardado la lista filtrada en un segundo estado.	Calcúlala en cada render a partir de los datos originales.
`sort()` o `reverse()` cambian la lista original.	Son métodos mutantes.	Usa `toSorted()` y `toReversed()`.
Aviso «Each child in a list should have a unique key».	Falta la `key` en el elemento del `map()`.	Añade `key={elemento.id}` (se explica en la UT 03.2).

Resumen y conclusiones

- Los elementos JSX son valores como cualquier otro: se pueden guardar en variables, devolver desde funciones y elegir según condiciones. En TypeScript, el tipo de «lo que se puede pintar» es `ReactNode`.
- El contenido se puede mostrar de forma condicional con `if` y *returns* tempranos, ternarios, `&&`, mapas de valores (`Record`), `switch` sobre uniones discriminadas o `<Activity>` si hay que conservar el estado.
- Con `&&`, la condición debe ser un booleano: si vale `0`, React lo pinta.
- Las uniones discriminadas modelan estados excluyentes (cargando / error / datos) y el tipo `never` en el `default` obliga a tratar todos los casos.
- Los arrays de JSX se pintan uno junto a otro. `map()` es el método estándar para convertir datos en JSX, y TypeScript infiere el tipo de cada elemento.
- Se filtra, ordena y agrupa **antes** de pintar, con valores derivados del estado (no con más estado). Prefiere los métodos no mutantes: `toSorted`, `toReversed`, `Object.groupBy`.
- Piensa siempre en el caso «no hay datos»: es un renderizado condicional más.
- En la UT 03.2 veremos cómo modificar estas listas desde el estado y por qué cada elemento necesita una `key`.

Preguntas de repaso

1. ¿Por qué `{mensajes.length && <Lista />}` puede mostrar un `0` en pantalla? ¿Cómo se corrige?
Respuesta: Porque `0` es «falso» pero es un valor que React sí pinta; `&&` devuelve el primer operando cuando es falso. Se corrige con una condición booleana: `mensajes.length > 0 && <Lista />`.

2. ¿Qué ventaja tiene tipar un mapa como `Record<Pais, string>` en lugar de usar un objeto sin tipar?
Respuesta: Obliga a definir todas las claves de `Pais` y permite indexar con un `Pais` sin error; si añades un país al tipo y olvidas el mapa, el código no compila.

3. ¿Qué diferencia hay entre `{visible && <Panel />}` y `<Activity mode={visible ? "visible" : "hidden"}><Panel /></Activity>`?
Respuesta: Con `&&` el componente se desmonta al ocultarlo y pierde su estado. Con `<Activity>` se oculta con `display: none`, conserva su estado (y el del DOM) y se destruyen sus efectos mientras está oculto.

4. ¿Para qué sirve asignar a una variable de tipo `never` en el `default` de un `switch` sobre una unión discriminada?
Respuesta: Para que el compilador avise si se añade un nuevo caso a la unión y no se ha tratado en el `switch` (comprobación de exhaustividad).

5. ¿Por qué es mejor `toSorted()` que `sort()` al trabajar con datos de React?
Respuesta: Porque `sort()` modifica el array original (mutación), mientras que `toSorted()` devuelve una copia ordenada y deja el original intacto.

Para saber más

- Documentación de React: *Renderizado condicional* (react.dev/learn/conditional-rendering).
- Documentación de React: *Renderizado de listas* (react.dev/learn/rendering-lists).
- Referencia de `<Activity>` (react.dev/reference/react/Activity) y notas de la versión React 19.2 (react.dev/blog/2025/10/01/react-19-2).
- TypeScript Handbook: *Narrowing* y uniones discriminadas (typescriptlang.org/docs/handbook/2/narrowing.html).